

MEGJEGYZÉSEK ÉS IMPRESSZUM

Ez a tankönyvprojekt az Oktatási és Kutatási Minisztérium **6 930 / 19.12.2025** számú rendeletével jóváhagyott, **10. osztályos Informatika iskolai tantervvvel** összhangban készült.

Gyermekbántalmazási vonal: 119

Gyermeksegélyvonal: 116 111

Szakmai És Szerkesztőségi Munkatársak

Tudományos Lektor 1: Anonim

Tudományos Lektor 2: Anonim

Tudományos Lektor 3: Anonim

Szerkesztőségi koordinátor: Anonim

Felelős szerkesztő: Anonim

Műszaki szerkesztő: Anonim

Nyomdai előkészítés: Anonim

Borítóterv / Design: Anonim

Kiadói És Megrendelési Elérhetőségek

Kiadó megnevezése: Anonim

Postacím / Terjesztés:

Telefonszám(ok):

Fax szám:

Weboldal:

ISBN szám:

10. OSZTÁLYOS INFORMATIKA TANTERV

(Matematika-Informatika Szak - Heti 4 óra / 144 óra/év)

1. Fejezet: Összetett Adatszerkezetek (Adatstruktúrák)

SSZ.	TÉMAKÖR / TANANYAG	SPECIFIKUS KOMPETENCIÁK & RÉSZLETEK	KAPCSOLT HTML / ÁLLOMÁNYOK	
1.1	Egydimenziós Tömbök és Halmazok (Set)	CS 1.1: Unió, metszet, különbség, tartozás vizsgálata, elemek elérése.	1.1_halmazok.html	
1.2	Kétdimenziós Tömbök (Mátrixok)	CS 1.1: Deklaráció, soronkénti/oszloponkénti bejárás, főátló és mellékátló, mátrixműveletek.	1.2_matrixok.html Mátrixok	
1.3	Karaktersorozatok (Stringek)	CS 1.1: Karakterelérés, összefűzés, keresés, lexikografikus összehasonlítás.	1.3_stringek.html	
1.4	Struktúrák / Rekordok (struct)	CS 1.1: Összetett entitások modellezése, mezők elérése, rekordok tömbje.	1.4_strukturak.html	
1.5	Asszociatív Struktúrák (Szótárak)	CS 1.1: Kulcs-érték párok tárolása, gyors keresés (hashing), szótárak használata.	1.5_szotarak.html	

2. Fejezet: Alprogramok, Függvények és Rekurzio

SSZ.	TÉMAKÖR / TANANYAG	SPECIFIKUS KOMPETENCIÁK & RÉSZLETEK	KAPCSOLT HTML / ÁLLOMÁNYOK	
2.1	Alprogramok és Modularitás	CS 2.1: Függvények, eljárások, visszatérési értékek, lokális és globális hatókörök.	2.1_alprogramok.html	
2.2	Paraméterátadási Mechanizmusok	CS 2.1: Érték szerinti és referencia szerinti (cím szerinti) paraméterátadás.	2.2_parameter.html	
2.3	Rekurzio és Rekurzív Algoritmusok	CS 2.2: Direkt/indirekt rekurzio, veremmemória (stack), Hanoi tornyai, Fibonacci.	2.3_rekurzio.html	

3. Fejezet: Algoritmikus Tervezési Stratégiák

SSZ.	TÉMAKÖR / TANANYAG	SPECIFIKUS KOMPETENCIÁK & RÉSZLETEK	KAPCSOLT HTML / ÁLLOMÁNYOK	
3.1	Keresés és Rendezés	CS 3.1: Bináris keresés, QuickSort, MergeSort és komplexitás.	3.1_rendezes.html	
3.2	Mohó Algoritmusok (Greedy)	CS 3.2: Helyi optimális döntések, érmeváltási feladat, hátizsák-probléma.	3.2_moho.html	
3.3	Dinamikus Programozás	CS 3.3: Részproblémák átfedése, optimális részstruktúra, memoizáció.	3.3_dinamikus.html	
3.4	Visszalépéses Keresés	CS 3.4: Backtracking elve, megoldási tér bejárása és vágási feltételek.	3.4_backtrack.html	

4. Fejezet: Szoftverfejlesztés, Modellek és Projektek

SSZ.	TÉMAKÖR / TANANYAG	SPECIFIKUS KOMPETENCIÁK & RÉSZLETEK	KAPCSOLT HTML / ÁLLOMÁNYOK	
4.1	Koncepcionális Modellek	CS 4.1: Szoftverfejlesztési életciklus, specifikációk meghatározása.	4.1_modellek.html	
4.2	Hozzáférési Modellek	CS 4.2: Szerepkör alapú hozzáférés (RBAC), adatvédelem.	4.2_hozzaferes.html	
4.3	Projektmunka és Tesztelés	CS 4.3: Egységtesztelés, hibakeresés (debugging), kód dokumentálás.	4.3_projekt.html	

FELFEDEZÉSES OKTATÁS A PROGRAMOZÁSBAN

Módszertani Útmutató pedagógusok számára • Interaktív logikai játékok alkalmazása

1. Alapelv: Miért a "Maguktól rájöjjenek" megközelítés?

Ahelyett, hogy a tanár készen átadná az algoritmusokat vagy a megoldási lépéseket, a vezérelt felfedezés (guided discovery) módszere arra ösztönzi a diákokat, hogy próbálgatással, teszteléssel és hibaelemzéssel maguk fedezzék fel a szabályokat.

A pedagógiai folyamat 4 fázisa:

- 1. **Fázis 1: Problémafelvetés** – Mutassunk be egy működő interaktív szimulációt (pl. folyadékrendező logikai játék). Ne magyarázzuk el a nyerési stratégiát, csak a szabályokat.
- 2. **Fázis 2: Hipotézisalkotás** – A diákok önállóan vagy párokban kísérleteznek, és feltevéseket gyártanak arról, hogy melyik lépés vezet eredményre.
- 3. **Fázis 3: Vezérelt Kérdések** – Ha elakadnak, ne adjuk meg a választ! Sokrátési módszerrel kérdezzünk: *"Mi történik, ha az üveg tetején lévő szín nem egyezik?"*
- 4. **Fázis 4: Generalizálás** – A tapasztalatok összegzése: a diákok saját szavaikkal fogalmazzák meg a logikai algoritmust.

 **Pedagógusi Tipp:** A kudarc itt nem hiba, hanem a tanulási folyamat szerves része. A visszavonás (undo) funkció biztonságarzertet ad a tanulóknak a merészebb kísérletekhez.

Gyakorlati Feladatlap: Kódkiegészítés & Logika

Töltsd ki az alábbi hiányos táblázatot és programkódrészletet a fizikai szimuláció szabályai alapján!

1. Feladat: Szabályok és Feltételek Táblázata

INTERAKCIÓ	FELTÉTEL (MIT KELL VIZSGÁLNI?)	KÖVETKEZMÉNY / EREDMÉNY
Üveg kiválasztása	tubes[idx].length > 0	[Írja be ide a diák: pl. Az üveg kiemelődik]
Folyadék átöntés	Célüveg üres VAGY felső szín egyezik	[Írja be ide a diák:]
Szint teljesítése	Minden üveg egyszínű és tele van (vagy üres)	Győzelmi ablak és jutalom érmék

2. Feladat: Kódkiegészítés (JavaScript Logika)

Egészítse ki az alábbi függvényt a hiányzó feltétellel, amely ellenőrzi, hogy egy adott üveg teljesen tele van-e azonos színű folyadékokkal!

```
function isTubeComplete(tube) {  
  if (tube.length < CAPACITY) return false;  
  let firstColor = tube[0];  
  for (let color of tube) {  
    // HIÁNYZÓ KÓD: Írja be a feltételt, ha eltér a szín!  
    if ([ Írja be ide a hiányzó feltételt ]) return false;  
  }  
  return true;  
}
```


